

代码片段 12.3 使用 JBD2 进行文件创建的例子

```
1 handle_t *handle;
2 // 原子操作：创建新文件
3 handle = jbd2_journal_start(journal, nblocks=8);
4
5 // 1. 标记 inode 为占用
6 bitmap_bh = read_inode_bitmap(sb, group);
7 jbd2_journal_get_write_access(handle, bitmap_bh);
8 set_bit(ino, bitmap_bh->b_data);
9 jbd2_journal_dirty_metadata(handle, bitmap_bh);
10
11 // 2. 初始化 inode
12 inode_bh = get_inode_bh(sb, ino);
13 jbd2_journal_get_write_access(handle, inode_bh);
14 init_inode(inode_bh);
15 jbd2_journal_dirty_metadata(handle, inode_bh);
16
17 // 3. 将目录项写入目录中
18 data_bh = get_data_page(dir_inode);
19 jbd2_journal_get_write_access(handle, data_bh);
20 add_dentry_to_data(page, filename, ino);
21 jbd2_journal_dirty_metadata(handle, data_bh);
22
23 jbd2_journal_stop(handle); // 结束原子操作
```

Ext4 的日志选项

使用日志会导致文件系统中的修改被写入存储设备两次——第一次写入日志区域，第二次写入数据原本应该写入的位置。当需要修改的数据量较大时，存储设备的两次写入开销会影响文件系统性能。为此，文件系统一般会在日志保证的崩溃一致性和性能之间进行权衡。例如，Ext4 文件系统对于元数据修改默认使用日志进行一致性保护，但对于文件数据部分，其提供 writeback、ordered 和 journal^[5-6] 三种选项（如表 12.2 所示）。

表 12.2 Ext4 文件系统中的日志选项

日志选项	数据是否使用日志保证	顺序要求	性能	一致性
writeback	否	无要求	最好	弱
ordered	否	数据在元数据之前写回	中等	中等
journal	是	数据和元数据均在日志中	较差	强

- 写回模式（Writeback Mode）。在该模式下，Ext4 对于文件数据部分的修改不进行日志记录，且对文件数据的写回没有特定的要求。在这种模式下，文件的数据块修改可以在任何时候写入磁盘中进行持久化。这种模式的一致性保证比较差。在发生崩溃的时候，有更大的概率发生不一致的情况。但是由于对写回顺序没有特殊要求，这种模式的性能往往是最优的。
- 顺序模式（Ordered Mode）。在该模式下，Ext4 对于文件数据部分的修改同样不进

行日志记录，但是对文件数据何时能够写回磁盘中进行持久化有特定的顺序要求。对于文件的数据和元数据修改，需要保证文件数据部分的修改在文件元数据被持久化前持久化到设备中。这一顺序的保证可以减少数据不一致的情况出现，增强文件系统的一致性保证。但对于跨多个存储块的数据写入，顺序模式无法保证这些数据持久化的原子性。因而如果在对文件中一段数据进行覆盖写入时发生崩溃，这一段数据可能一半是新的，另一半是旧的。该模式是 Ext4 中的默认模式，也是一种性能和一致性保证之间的权衡。

- 日志模式 (Journal Mode)。在该模式下，Ext4 文件中的数据和元数据修改均使用日志进行保护。这是一种一致性保证很强的模式，但是要求文件数据在日志中和原位置上进行两次写入。因此，对于产生大量文件数据修改的场景来说，这种模式会带来不小的性能开销和不必要的写入操作。

12.3.4 讨论和小结

小思考

使用日志有哪些优点和缺点？什么情况下适合使用日志进行一致性保证？

日志是一种保证原子更新和崩溃一致性的常用方法。通过日志记录，可以保证任意位置、任意数量操作的原子完成，因而应用范围比较广泛，特别适用于修改位置比较分散的场景。然而，使用日志同样会带来一些问题。首先，日志要求所有的修改先在日志中进行记录，再将修改应用到其原有位置上。这样所有的修改都执行了两次：一次在日志中，另外一次在原位置上。因此，对于数据修改较多的场景，使用日志保证原子性可能会带来很大的写入开销。其次，日志的原子性保证依赖于日志恢复，因此在发生崩溃并重启后，需要首先进行日志恢复。只有日志恢复完毕之后，文件系统才能开始处理新的请求。若日志恢复时间较长，整个文件系统将长时间处于不可用状态，进而影响操作系统或应用程序的启动时间。最后，在使用日志时，需要为日志预留一定的空间。在使用日志时，这部分空间会被反复写入和修改。对于闪存等容易被写穿的存储介质来说，日志区域将导致比较严重的磨损问题。

12.4 原子更新技术：写时拷贝

本节主要知识点

- 写时拷贝技术如何保证多处数据的原子更新？
- 文件系统如何使用写时拷贝技术保证崩溃一致性？

12.4.1 写时拷贝的原理

写时拷贝是另外一种能够保证多处修改原子更新的技术。当多处修改最终能转换为一个原子操作（如通过修改指针进行原子更新）时，可以使用写时拷贝保证多处修改的原子性。使用写时拷贝修改数据时，需要首先对原数据进行一次拷贝，并在拷贝出来的数据副本上进行修改。此后，写时拷贝通过修改指向原数据的指针，使其指向新数据（即修改后的数据副本），从而让副本中的数据修改生效。对于需要原子完成的多处修改，写时拷贝可能需要修改多处指针。对于这些指针的修改，如果无法在一次原子操作内完成，往往需要继续使用写时拷贝的方式进行，直至所有修改可以在一次原子操作内完成。

写时拷贝经常被用在树形数据结构中。由于整个树形数据结构只有一个树根，不论想要修改树中的多少个节点，最终都能将修改转变为对根节点的原子操作，以保证整个树上的所有修改均是原子完成的。

图 12.3 给出了树形数据结构上写时拷贝技术的一个示例。最初，树中有 A 到 F 共六个节点。我们需要原子地修改 C 和 E 两个节点中的数据。在进行修改时，我们并不直接修改 C 和 E 节点中的数据，而是分别为 C 和 E 创建拷贝，并在拷贝中进行修改，生成 C' 和 E' 两个节点。注意，此时无法根据树根访问到修改后的 C' 和 E'，因此这两处修改并未生效。此后，为了让修改生效，需要进一步写时拷贝节点 B、D 以及树根 A，最终通过修改指向树根的指针来保证 C' 和 E' 两处修改生效的原子性。旧的节点在操作完成后可以被回收以释放空间。

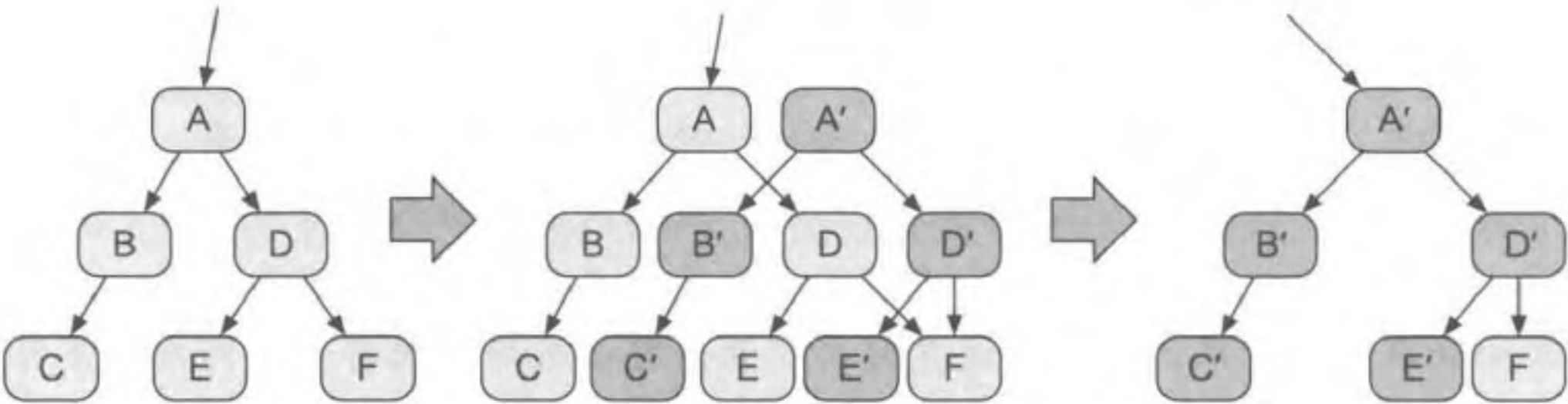


图 12.3 通过写时拷贝技术保持多处修改的原子性

原子更新点。在上述例子中，原子更新的完成可以被归结到一个原子操作上，即对指向树根的指针的修改。在此操作完成之前，由于整个数据结构的原有数据并未被修改，因此所有的修改均未生效（即无法通过树根指针访问到）。在此操作完成之后，所有的修改同时生效。这个原子操作被称为原子更新点。

当使用写时拷贝技术在存储设备上修改数据结构时，原子更新点即为数据的原子持久化点。对于图 12.3 中的例子来说，当 C'、E'、B'、D' 和 A' 的数据已经持久化之后，可以对树根指针进行修改，并将修改持久化。树根指针的持久化是整个操作的原子持久化点。在树根指针的修改持久化之前，无论对于 C'、E'、B'、D' 和 A' 的修改是否已经被持久化，在发生掉电或崩溃后，系统都无法访问到这些修改，因此不会产生一致性问题。

而一旦树根修改持久化完成，在 C'、E'、B'、D' 和 A' 上的所有修改也会同时生效。由于这些修改在此前已经确保持久化完毕，因此掉电或崩溃后，系统能访问到完整的修改后的数据结构，不会产生一致性问题。

空间回收。在使用写时拷贝技术进行原子更新后，需要对保存原有数据的结构进行回收。在图 12.3 中，可以被释放的节点包括 A、B、C、D、E。空间回收只是为了防止空间浪费，保证后续的分配能够顺利完成。因此若空闲空间充足，空间回收可以延后或者异步进行。

12.4.2 写时拷贝在文件系统中的应用

写时拷贝技术经常被用于保证文件数据的一致性。图 12.4 给出了一个使用写时拷贝保证文件数据一致性的示例。图中的文件 inode 中保存了多个数据块和一个索引块的指针。为了方便表述，我们将图中给出的数据块从 A 到 F 进行了标号，其中相邻数据块的逻辑地址是相连的。当一个文件写请求跨越了多个数据块的边界时，文件系统需要发起多次存储请求来完成数据修改^①。若不加任何保障，在进行修改时发生崩溃可能会造成文件数据的不一致。例如，当一个文件写请求刚好跨越数据块 C 和 D 的边界，而在文件系统刚刚修改完数据块 C 之后，系统发生崩溃，在重新启动计算机后，在数据块 C 中的数据为文件写请求之后的新数据，而在数据块 D 中的数据为文件写请求之前的旧数据，文件数据出现了不一致的情况。



图 12.4 利用写时拷贝技术修改文件数据

inode 的结构与图 12.3 中的树状结构是相似的，因此，可以使用写时拷贝保证跨数据块写请求的原子持久化。具体来说，我们会将数据块 C 和 D 进行拷贝，并在拷贝副本上进行修改。由于指向数据块 C 的指针保存在 inode 中，而指向数据块 D 的指针保存在索引块中，我们需要继续对索引块进行写时拷贝。最终，需要修改的指向数据块 C 的指针和指向索引块的指针均保存在 inode 结构中。由于 inode 结构通常小于存储设备的块大小，通过原子更新 inode 结构，我们可以原子地持久化文件数据块 C 和 D 上的数据修改。

① 此处假设图中的每个框对应一个存储设备的原子访问单元（即块）。

由于所有文件数据访问均从 inode 结构出发，因此一个文件中任何位置上的修改，均可通过写时拷贝技术进行原子更新和持久化。

小知识：使用写时拷贝技术保护整个文件系统

除了用来保证文件数据的一致性之外，写时拷贝技术还可以用来保证整个文件系统的一致性^[7]。如在 BtrFS 中，整个存储结构由一棵树组成，可以通过写时拷贝技术保证多个位置上修改的原子完成。

12.4.3 写时拷贝的问题与优化

在上述例子中，我们只希望修改节点 C 和 E 中的部分数据，却导致节点 B、D、A 也被修改。这表现出了写时拷贝的两个问题：更新传递问题和写放大问题。

在树形数据结构中，写时拷贝往往需要更新到根节点才会停止。在图 12.3 中，我们修改节点 C 和 E 后，需要继续修改节点 B、D 和 A（根节点）。这种修改一些节点后，还需要进一步修改数据结构中的其他节点以保证原子更新的问题，被称为更新传递问题。

解决更新传递问题的一种常见方法是提前进行原子更新。对于树形数据结构来说，并非只有指向树根的指针才可以被原子更新；在上述例子中，原子更新的粒度为一个节点。若在更新传递的过程中，所有要进行的修改被一个原子更新粒度所覆盖，则可以直接原地进行修改，无须继续传递更新。例如在图 12.3 中，在节点 A 中保存了指向节点 B 和 D 的指针。由于节点 A 可以被原子更新，我们可以通过一次原地更新操作，原子地修改节点 A 中指向节点 B 和 D 的指针，让它们指向节点 B' 和 D'，从而避免继续使用写时拷贝技术，停止更新传递。

写放大问题是指实际修改数据量大于用户要修改的数据量的情况。如在应用以 4KB 内存页为粒度的写时拷贝技术时，若想修改某个页中的 1 字节数据，我们不得不拷贝整个 4KB 内存页中的所有数据，因此数据修改量从 1 字节被放大到 4KB。写时拷贝技术中的更新传递会使写放大问题更加严重。

当要修改的数据覆盖了拷贝操作的粒度时，可以不拷贝原有数据，直接写入新的数据，从而在一定程度上缓解写放大问题。例如在图 12.3 中，拷贝粒度为一个节点，且节点 E 中的所有数据均需要被修改，在分配 E' 后，即使我们将 E 中的数据拷贝到 E'，这些数据也会被新数据全部覆盖。因此，在这种情况下，可以省略写时拷贝中的拷贝操作，从而减少写入数据量。此外，前面提到的提前原子更新的方法，也可以在一定程度上减轻写放大问题。

12.4.4 讨论和小结

写时拷贝技术对数据结构有一定的要求，当所有的修改最终能够变成一个原子修改

时,才可以使用写时拷贝技术。此外,写时拷贝技术的使用与原子更新粒度和拷贝粒度有关。当修改的数据量远大于拷贝粒度时,往往只有数据头部和尾部真正需要进行拷贝操作,而中间部分可以根据前文提到的方法省略拷贝操作,直接使用新数据写入新分配的节点。这种情况下,写时拷贝技术产生的写放大相对较小。而当数据修改量小于拷贝操作的粒度时,写时拷贝技术造成的写放大会非常严重,在块设备上,即使每次仅修改 1 字节数据,也需要按照块粒度进行数据拷贝。因此,是否适合使用写时拷贝,需要结合原子更新粒度、拷贝粒度和数据修改量综合考虑。

12.5 Soft updates

本节主要知识点

- 为什么要提出 Soft updates ?
- Soft updates 技术如何保证崩溃一致性?
- Soft updates 技术如何解决环形依赖问题?
- Soft updates 为何无须进行恢复?

虽然使用原子更新技术能够较好地保证文件系统的崩溃一致性,但是在其他方面依然存在一些问题。首先,这些原子更新技术对存储布局和文件系统设计存在一定的要求。例如使用日志时,需要在存储空间中为日志预留空间,同时需要保存日志相关的各种元数据;在写时拷贝技术中,为了方便地通过修改指针原子更新结构内容,每个结构的位置一般由上一级结构中的指针决定,而不能处于固定位置。这限制了文件系统设计的灵活性和可拓展性。此外,使用这些原子更新技术需要在修改过程中对存储设备进行额外的写入操作。例如文件系统使用日志时,需要先将修改的日志记录到存储设备上;在使用写时拷贝时,需要将原来的数据进行一次拷贝操作。这些额外的写入操作反而影响了文件系统的性能。

考虑到这些问题,原子更新技术的使用条件对于一些文件系统来说有些过于严格,影响了文件系统的设计和性能。那么除了原子更新技术之外,有没有其他方法可以保证崩溃一致性呢?

我们不妨回到 12.2.1 节提到的同步写入问题重新进行考虑。为了保证多个修改的持久化顺序,同步写入需要等待前一个修改持久化到存储设备之后,才能开始进行下一个修改。实际上,为了保证崩溃一致性,文件系统仅需保证修改被持久化的顺序,并不需要同步等待每个修改的持久化完成。如果存在某个机制能够保证这些修改的持久化顺序,那么文件系统在内存结构中执行完一个修改之后,无须等待其持久化完成便可以开始处理下一个修改。在这种情况下,文件系统能够以接近内存的性能处理文件请求,并保证